

INF 662 · DESARROLLO DE APLICACIONES MÓVILES

Guía de Laboratorio N.º 10

MultiProvider y Selector: varios estados y rendimiento

MATERIA INF 662 — Desarrollo de Aplicaciones Móviles**SEMESTRE** 6.º · Mención Ing. de Software**TEMA** Varios providers y optimización**DURACIÓN** 5 horas (1 sesión de laboratorio)**CAPÍTULO** Cap. 4 — Manejo del estado con Provider**MODALIDAD** Laboratorio · individual**DOCENTE** M. Sc. Lic. Huáscar Fedor Gonzales Guzmán**REQUISITO** Guía de Laboratorio N.º 9**NOTA · Continúa desde la Guía N.º 9**

Partimos del proyecto `carrito_app` con su `CarritoModel`. Ahora agregaremos un **segundo estado** (el tema claro/oscuro) con `MultiProvider`, y usaremos `Selector` para que los widgets se reconstruyan solo cuando cambia el dato que realmente usan.

01 Objetivos

Objetivo general. Gestionar varios estados independientes con `MultiProvider` y optimizar las reconstrucciones de la interfaz con `Selector`, ampliando la app de carrito con un tema conmutable.

Objetivos específicos:

- Crear un segundo `ChangeNotifier` para el tema (claro/oscuro).
- Registrar varios providers a la vez con `MultiProvider`.
- Aplicar el estado del tema en `MaterialApp` con `ThemeMode`.
- Optimizar reconstrucciones con `Selector` frente a `Consumer`.
- Reconocer cuándo una parte de la UI se reconstruye de más y corregirlo.

02 Marco teórico

DEFINICIÓN · MultiProvider

Cuando la app necesita más de un estado global, `MultiProvider` permite registrar varios providers en una sola lista, en lugar de anidarlos manualmente uno dentro de otro.

DEFINICIÓN · Selector

`Consumer` se reconstruye ante **cualquier** cambio del modelo. `Selector<T, R>` observa solo una parte `R` del estado y reconstruye únicamente cuando esa parte cambia. Reduce reconstrucciones innecesarias.

DEFINICIÓN · ThemeMode

`MaterialApp` acepta `theme`, `darkTheme` y `themeMode`. Cambiando `themeMode` entre `ThemeMode.light` y `ThemeMode.dark` se alterna la apariencia de toda la app.

03 Requisitos previos

Requisito indispensable. Haber completado la Guía N.º 9 (Provider + ChangeNotifier) y contar con el proyecto del carrito.

Conceptos que se reutilizan:

ChangeNotifier

context.watch / read

Consumer

NOTA · Trabajaremos sobre el carrito

Esta guía amplía `carrito_app`. Si no lo conservas, puedes recrear un `CarritoModel` mínimo (agregar/quitar/total) siguiendo la Guía 9 antes de empezar.

04 Desarrollo · Parte A — Un segundo estado: el tema

1 Crear el TemaModel

1. En `lib/main.dart`, junto al `CarritoModel`, crea un segundo `ChangeNotifier` que guarde si el modo oscuro está activo:

LIB/MAIN.DART · DART

```
class TemaModel extends ChangeNotifier {
  bool _oscuro = false;

  bool get oscuro => _oscuro;

  ThemeMode get modo =>
    _oscuro ? ThemeMode.dark : ThemeMode.light;

  void alternar() {
    _oscuro = !_oscuro;
    notifyListeners();
  }
}
```

NOTA · Estados independientes

`TemaModel` y `CarritoModel` no se conocen entre sí: cada uno gestiona su propia responsabilidad. Separar estados por tema facilita mantener la app.

2 Registrar ambos con MultiProvider

1. Reemplaza el `ChangeNotifierProvider` único por un `MultiProvider` con los dos modelos:

LIB/MAIN.DART · DART

```
void main() {
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(
          create: (_) => CarritoModel(),
        ),
        ChangeNotifierProvider(
          create: (_) => TemaModel(),
        ),
      ],
      child: const MiApp(),
    ),
  );
}
```

NOTA · Orden y legibilidad

Con `MultiProvider` la lista de providers queda plana y legible. Agregar un tercer estado en el futuro es solo añadir un elemento más a `providers`.

3 Aplicar el tema en MaterialApp

1. En el widget `MiApp`, escucha el `TemaModel` y pásale su modo a `MaterialApp`:

LIB/MAIN.DART · DART

```
class MiApp extends StatelessWidget {
  const MiApp({super.key});

  @override
  Widget build(BuildContext context) {
    final tema = context.watch<TemaModel>();
    return MaterialApp(
      title: 'Carrito',
      theme: ThemeData.light(),
      darkTheme: ThemeData.dark(),
      themeMode: tema.modos,
      home: const PantallaProductos(),
    );
  }
}
```

4 Botón para alternar el tema

1. En el `AppBar` de la pantalla de productos, agrega una acción que use `context.read` para alternar el tema:

DART · EN ACTIONS DEL APPBAR

```

IconButton(
  icon: Icon(
    context.watch<TemaModel>().oscuro
      ? Icons.light_mode
      : Icons.dark_mode,
  ),
  onPressed: () =>
    context.read<TemaModel>().alternar(),
),

```

2. Ejecuta: al pulsar el botón, toda la app cambia entre claro y oscuro, y el carrito sigue funcionando igual.

05 Desarrollo · Parte B — Optimizar con Selector

5 Detectar reconstrucciones de más

1. Supón que en el `AppBar` quieres mostrar solo la **cantidad** de ítems del carrito. Con `context.watch<CarritoModel>()` ese widget se reconstruye ante cualquier cambio del carrito, aunque la cantidad no varíe.
2. Para comprobarlo, puedes imprimir en consola dentro del `build` del widget con `debugPrint` y observar cuántas veces se ejecuta.

6 Usar Selector para la cantidad

1. Envuelve solo el contador con un `Selector` que observe únicamente `cantidad`. Así se reconstruye solo cuando ese número cambia:

DART · EN EL APPBAR

```

Selector<CarritoModel, int>(
  selector: (context, carrito) => carrito.cantidad,
  builder: (context, cantidad, child) {
    return Text(
      'Items: ' + cantidad.toString(),
      style: const TextStyle(fontSize: 18),
    );
  },
),

```

EJEMPLO · Cómo trabaja Selector

El primer parámetro de tipo (`CarritoModel`) es el modelo observado; el segundo (`int`) es el dato seleccionado. Provider compara el valor anterior y el nuevo: si son iguales, **no reconstruye**.

7 child para lo que no cambia

1. Si dentro del `builder` hay un widget que nunca cambia, pásalo por el parámetro `child` para que no se reconstruya con cada actualización:

DART · CHILD REUTILIZABLE

```
Selector<CarritoModel, int>(
  selector: (context, carrito) => carrito.cantidad,
  child: const Icon(Icons.shopping_cart),
  builder: (context, cantidad, child) {
    return Row(
      children: [
        child!,
        const SizedBox(width: 4),
        Text(cantidad.toString()),
      ],
    );
  },
),
```

NOTA · El parámetro child

El `child` se construye una sola vez y se reutiliza en cada reconstrucción del `builder`. Es una micro-optimización útil para partes fijas (iconos, etiquetas).

06 Consumer vs. Selector

Ambos consumen el estado; se diferencian en cuándo reconstruyen:

Aspecto	Consumer	Selector
Reconstruye	Ante cualquier cambio	Solo si cambia lo seleccionado
Ideal para	Usar todo el modelo	Usar un dato puntual
Costo	Más reconstrucciones	Menos reconstrucciones

ADVERTENCIA · No optimices de más

Usa `Selector` cuando notes reconstrucciones costosas o listas grandes. Para UI sencilla, `Consumer` o `context.watch` son más legibles. Primero claridad, luego rendimiento.

07 Verificación de resultados y entregable

Marca cada punto como verificación de que completaste la práctica:

- ☐ TemaModel (ChangeNotifier) alterna entre claro y oscuro.
- ☐ Los dos estados se registran con MultiProvider.
- ☐ MaterialApp aplica el tema con themeMode.
- ☐ El botón del AppBar cambia el tema de toda la app.
- ☐ El contador de ítems usa Selector y observa solo la cantidad.
- ☐ El carrito sigue funcionando junto con el nuevo tema.

EJEMPLO · Qué entregar

El archivo `lib/main.dart` terminado y un documento (PDF o Word) con: (1) captura de la app en modo claro; (2) captura en modo oscuro; (3) captura del carrito funcionando; y (4) **reto personal**: agrega un tercer estado con `MultiProvider` (por ejemplo, un contador de favoritos) y muéstralo con su propio `Selector`. Descríbelo en 3–5 líneas.

08 Cuestionario

1. ¿Qué ventaja aporta `MultiProvider` frente a anidar varios `ChangeNotifierProvider`?
2. ¿En qué se diferencia `Selector` de `Consumer` respecto a las reconstrucciones?
3. ¿Qué representan los dos tipos genéricos en `Selector<T, R>`?
4. ¿Para qué sirve el parámetro `child` dentro de `Selector` o `Consumer`?
5. ¿Por qué conviene priorizar la claridad y optimizar con `Selector` solo cuando es necesario?

09 Rúbrica de evaluación

Criterio	Puntaje
TemaModel y alternancia claro/oscuro	25 %
<code>MultiProvider</code> con los dos estados	20 %
Tema aplicado con <code>themeMode</code> en <code>MaterialApp</code>	15 %
Uso correcto de <code>Selector</code> (cantidad)	25 %
Cuestionario resuelto	15 %

10 Referencias

- Paquete `provider` — `MultiProvider` y `Selector` (pub.dev). pub.dev/packages/provider
- Flutter — Simple app state management. [docs.flutter.dev · state-mgmt/simple](https://docs.flutter.dev/state-mgmt/simple)
- Flutter — Themes and `ThemeMode`. docs.flutter.dev/cookbook/design/themes

Contenido basado en la documentación oficial de Flutter (v3.44) y el paquete `provider` (^6.x). Elaborado para INF 662 — UATF.